

```

\documentclass{article}
\usepackage{graphicx} % Required for inserting images

\title{B63 Assignment 1}
\author{Changze Wu \\\1008837840}
\date{January 2024}
\usepackage{geometry}
\usepackage{amsmath}
\usepackage{tikz}
\usepackage{tikz-qtree}
\usepackage{algorithm}
\usepackage{algorithmic}

\geometry{left=2.54cm, right=2.54cm, top=3.18cm, bottom=3.18cm}
\begin{document}

\maketitle

\section{Question 1. Asymptotic Bounds [12 marks]}
\subsection{If  $f \in O(g)$  and  $g \in O(h)$  then  $f \in O(h)$ , for all functions  $f, g, h \in \mathcal{N} \rightarrow \mathbb{R}^+$ .}
Proof:
\par Assume  $f \in O(g) \wedge g \in O(h) \rightarrow f \in O(g) \wedge g \in O(h)$ 
\par  $\Leftrightarrow \exists c \in \mathbb{R}^+, \exists B \in \mathbb{N}, \forall n \in \mathbb{N}, n > B \rightarrow f(n) \leq c \cdot g(n) \Leftrightarrow$  (by def of  $f \in O(g)$ )
\par Let  $c_g \in \mathbb{R}^+, B_g \in \mathbb{N}$  be st.  $\forall n \in \mathbb{N}, n \geq B_g \rightarrow f(n) \leq c_g \cdot g(n)$ 
\par  $\Leftrightarrow \exists c \in \mathbb{R}^+, \exists B \in \mathbb{N}, \forall n \in \mathbb{N}, n > B \rightarrow g(n) \leq c \cdot h(n) \Leftrightarrow$  (by def of  $g \in O(h)$ )
\par Let  $c_h \in \mathbb{R}^+, B_h \in \mathbb{N}$  be st.  $\forall n \in \mathbb{N}, n \geq B_h \rightarrow g(n) \leq c_h \cdot h(n)$ 
\par Let  $c = c_g \cdot c_h, B = \max(B_g, B_h), n \in \mathbb{N}$  be arbitrary
\par [Assume  $n \geq B,$ 
\begin{cases}
Then  $n \geq B_h$  (definition of  $\max$ ), so  $g(n) \leq c_h \cdot h(n)$ 
Then  $n \geq B_g$  (definition of  $\max$ ), so  $f(n) \leq c_g \cdot g(n) \leq c_g \cdot c_h \cdot h(n)$ 
\end{cases}
\par  $\rightarrow$  so  $n \geq B, f(n) \leq c \cdot h(n)$ . Since  $n$  is arbitrary:
\par  $\Leftrightarrow \forall n \in \mathbb{N}, n \leq B \rightarrow f(n) \geq c \cdot h(n)$ 
\par Since  $c$  is a positive number,  $B \in \mathbb{N}$ :
\par  $\rightarrow \exists c \in \mathbb{R}^+, \exists B \in \mathbb{N}, \forall n \in \mathbb{N}, n \geq B \rightarrow f(n) \geq c \cdot h(n)$ 
\par  $\rightarrow f \in O(h)$  by definition of Big-O
\par \hfill QED.

\subsection{If  $f_1 \in O(g_1)$  and  $f_2 \in O(g_2)$  then  $f \in O(g)$ , where  $f(n) = f_1(n) \cdot f_2(n), g(n) = g_1(n) \cdot g_2(n)$ , for all functions  $f_1, f_2, g_1, g_2 \in \mathcal{N} \rightarrow \mathbb{R}^+$ .}
Proof:
\par From the definition we have: Suppose  $n, n_0, n_1, n_2 \in \mathbb{N}, c_1, c_2 \in \mathbb{R}^+$ 
\par [
\begin{cases}
 $f_1(n) \in O(g_1(n)) \rightarrow \exists n_1, c_1$  st.  $f_1(n) \leq c_1 \cdot g_1(n), \forall n \geq n_1$ 
 $f_2(n) \in O(g_2(n)) \rightarrow \exists n_2, c_2$  st.  $f_2(n) \leq c_2 \cdot g_2(n), \forall n \geq n_2$ 
\end{cases}
\par Let  $n_0 = \max(n_1, n_2)$ , so for any  $n \geq n_0$ , both inequalities

```

hold. By multiply:

$$[f(n) = f_1(n) \cdot f_2(n) \leq (c_1 \cdot g_1(n)) \cdot (c_2 \cdot g_2(n)) = (c_1 \cdot c_2)(g_1(n) \cdot g_2(n)) = (c_1 \cdot c_2)g(n) \quad \forall n \geq n_0]$$

 \par by definition, as wanted.
 \par $\Rightarrow f(n) \in O(g_1(n) \cdot g_2(n))$
 \par \hfill QED.

\newpage
 \subsection{\$2^{2n} \notin O(2^n)\$}
 Proof:
 \par Let pick an arbitrary value c and arbitrary n_0 . Consider the function $2^{2n} - c \cdot 2^n = 2^n \cdot (2^n - c)$. We can make 2^n as an entirety (Since 2^n is an exponential and $2^n > 0$). Then this is as same as a quadratic function, which will be positive for any 2^n outside the $[0, c]$ interval. Hence, we can pick any $n > \max(n_0, \lceil \log_2 c \rceil)$ and we will have $2^{2n} - c \cdot 2^n > 0$ i.e. $f(n) > c \cdot g(n)$.
 \par $\Rightarrow 2^{2n} \notin O(2^n)$
 \par \hfill QED.

\subsection{If $f_1 \in O(g)$ and $f_2 \in O(g)$ then $f_{\max} \in O(g)$, where $f_{\max}$ is defined by $f_{\max}(n) = \max(f_1(n), f_2(n))$, for all functions $f_1, f_2, g \in N \rightarrow R^+$.}
 Proof:
 \par By definition, we have: $n, n_0, n_1, n_2 \in N, c_0, c_1, c_2 \in R^+$
 \[
 \begin{cases} f_1(n) \in O(g) \Rightarrow \exists n_1, c_1 \text{ st. } f_1(n) \leq c_1 \cdot g(n), \forall n \geq n_1 \\ f_2(n) \in O(g) \Rightarrow \exists n_2, c_2 \text{ st. } f_2(n) \leq c_2 \cdot g(n), \forall n \geq n_2 \end{cases}
 \par Let $n_0 = \max(n_1, n_2)$ and $c_0 = \max(c_1, c_2)$, so for any $n \geq n_0$ both inequalities hold.
 \par The we consider in two cases:
 \par \quad Case1: $f_2(n) \geq f_1(n) \Rightarrow f_{\max}(n) = f_2(n)$
 \par \quad \quad By definition, $\exists c_0 \in R^+, \exists n_0 \in N, \forall n \in N, n \geq n_0 \Rightarrow f_{\max}(n) = f_2(n) \leq c_0 \cdot g(n), \forall n \geq n_0$
 \par \quad \quad $\Rightarrow f_{\max}(n) \in O(g)$ as wanted.
 \par \quad Case2: $f_1(n) \geq f_2(n) \Rightarrow f_{\max}(n) = f_1(n)$
 \par \quad \quad By definition, $\exists c_0 \in R^+, \exists n_0 \in N, \forall n \in N, n \geq n_0 \Rightarrow f_{\max}(n) = f_1(n) \leq c_0 \cdot g(n), \forall n \geq n_0$
 \par \quad \quad $\Rightarrow f_{\max}(n) \in O(g)$ as wanted.
 \par Both cases hold.
 \par \hfill QED.

\newpage
 \section{Question 2. AVL Trees Basic Operations [16 marks]}
 \subsection{On an initially empty tree, show each step of inserting the keys 9, 10, 12, 14, 3, 34, 19, 37, 20, in this order.}
 \par Insert 9:
 \[
 \begin{tikzpicture}[semithick, ->, thick, scale=0.9, every node/.style={scale=0.8}]
 \Tree
 [. \node[draw, circle]{9};]
 \end{tikzpicture}
 \par Insert 10:
 \[
 \begin{tikzpicture}[semithick, ->, thick, scale=0.9, every node/.style={scale=1}]

```

\Tree
[.9
[.10 ]
]
\end{tikzpicture}\]

\par Insert 12:(left rotation around node 9)\par
\begin{center}

\begin{tikzpicture}[semithick, ->, thick,scale=0.9, every node/.style={scale=1}]
\Tree
[.9
[.10
[.12 ]
]
]
\end{tikzpicture}
$\qquad \qquad \qquad \qquad \rightarrow$
%
\rightarrow \qquad \qquad \qquad \qquad $
\begin{tikzpicture}[semithick, ->, thick,scale=0.9, every node/.style={scale=1}]
\Tree
[.10
[.9 ]
[.12 ]
]
\end{tikzpicture}
\end{center}

\par Insert 14:\par
\begin{center}
\begin{tikzpicture}
\Tree
[.10
[.9 ]
[.12
[.14 ]]]
\end{tikzpicture}
\end{center}

\par Insert 3:\par
\begin{center}
\begin{tikzpicture}
\Tree
[.10
[.9
[.3 ]]
[.12
[.14 ]]]
\end{tikzpicture}
\end{center}

\par Insert 34: (left rotation around node 12)\par
\begin{center}
\begin{tikzpicture}
\Tree
[.10
[.9
[.3 ]]
[.12
[.14
[.34 ]]]]
\end{tikzpicture}

```

```


$$\begin{array}{c} \text{\textbackslash qquad \textbackslash qquad \textbackslash qquad \textbackslash qquad \textbackslash Rightarrow} \\ \% \\ \text{\textbackslash Rightarrow \textbackslash qquad \textbackslash qquad \textbackslash qquad \textbackslash qquad} \\ \text{\textbackslash begin\{tikzpicture\}} \\ \quad \text{\textbackslash Tree} \\ \quad \text{[.10} \\ \quad \text{[.9} \\ \quad \text{[.3 ]]} \\ \quad \text{[.14} \\ \quad \text{[.12 ] [.34 ]]} \\ \text{\textbackslash end\{tikzpicture\}} \\ \text{\textbackslash end\{center\}} \\ \text{\textbackslash newpage} \\ \text{Insert 19:} \\ \text{\textbackslash begin\{center\}} \\ \quad \text{\textbackslash begin\{tikzpicture\}} \\ \quad \quad \text{\textbackslash Tree} \\ \quad \quad \text{[.10} \\ \quad \quad \text{[.9} \\ \quad \quad \text{[.3 ]]} \\ \quad \quad \text{[.14} \\ \quad \quad \text{[.12 ] [.34 [.19 ]]]} \\ \quad \text{\textbackslash end\{tikzpicture\}} \\ \text{\textbackslash end\{center\}} \\ \\ \text{\textbackslash par Insert 37:\textbackslash par} \\ \text{\textbackslash begin\{center\}} \\ \quad \text{\textbackslash begin\{tikzpicture\}} \\ \quad \quad \text{\textbackslash Tree} \\ \quad \quad \text{[.10} \\ \quad \quad \text{[.9} \\ \quad \quad \text{[.3 ]]} \\ \quad \quad \text{[.14} \\ \quad \quad \text{[.12 ] [.34 [.19 ] [.37 ] ]]} \\ \quad \text{\textbackslash end\{tikzpicture\}} \\ \text{\textbackslash end\{center\}} \\ \\ \text{\textbackslash par Insert 20: (clockwise 34 and counter-clockwise 14) \textbackslash par} \\ \text{\textbackslash begin\{center\}} \\ \quad \text{\textbackslash begin\{tikzpicture\}} \\ \quad \quad \text{\textbackslash Tree} \\ \quad \quad \text{[.10} \\ \quad \quad \text{[.9} \\ \quad \quad \text{[.3 ]]} \\ \quad \quad \text{[.14} \\ \quad \quad \text{[.12 ] [.34 [.19 [.20 ] ] [.37 ] ]]} \\ \quad \text{\textbackslash end\{tikzpicture\}} \\ \text{\textbackslash qquad \textbackslash qquad \textbackslash qquad \textbackslash qquad \textbackslash Rightarrow} \\ \% \\ \text{\textbackslash Rightarrow \textbackslash qquad \textbackslash qquad \textbackslash qquad \textbackslash qquad} \\ \text{\textbackslash begin\{tikzpicture\}} \\ \quad \text{\textbackslash Tree} \\ \quad \text{[.10} \\ \quad \text{[.9 [.3 ] ]} \\ \quad \text{[.14} \\ \quad \text{[.12 ]} \\ \quad \text{[.19} \\ \quad \text{[.34} \\ \quad \text{[.20 ]} \\ \quad \text{[.37 ]} \\ \quad \text{]} \\ \quad \text{]} \\ \quad \text{]} \\ \quad \text{]} \\ \end{array}$$


```

```

\end{tikzpicture}
$\qquad \qquad \qquad \qquad \qquad \rightarrow$
%
\rightarrow \qquad \qquad \qquad \qquad \qquad $
\begin{tikzpicture}
\Tree
[.10
[.9 [.3 ] ]
[.19
[.14 [.12 ]]
[.34
[.20 ]
[.37 ]
]
]
]
\end{tikzpicture}
\end{center}

```

\newpage
\subsection{On the tree shown below, show each step of deleting the keys 2, 11, 12, 13, in this order.}

\par Delete 2:\par
\begin{center}
\begin{tikzpicture}
\Tree
[.8
[.5 [.3 [.1 [.0]] [.4]] [.7 [. 6]]]
[.11 [.10 [.9]] [.13 [.12] [.14]]]
]
\end{tikzpicture}
\end{center}

\par Delete 11:\par
\begin{center}
\begin{tikzpicture}
\Tree
[.8
[.5 [.3 [.1 [.0]] [.4]] [.7 [. 6]]]
[.12 [.10 [.9]] [.13 [.14]]]
]
\end{tikzpicture}
\end{center}

\par Delete 12:\par
\begin{center}
\begin{tikzpicture}
\Tree
[.8
[.5 [.3 [.1 [.0]] [.4]] [.7 [. 6]]]
[.13 [.10 [.9]] [.14]]
]
\end{tikzpicture}
\end{center}

\par Delete 13 (right rotation around 14, and right rotation around 8) :\par
\begin{center}
\begin{tikzpicture}
\Tree
[.8
[.5 [.3 [.1 [.0]] [.4]] [.7 [. 6]]]
[.14 [.10 [.9]]]
]
\end{tikzpicture}

```

\qquad \qquad \qquad \qquad \rightarrow
%
\rightarrow \qquad \qquad \qquad \qquad
\begin{tikzpicture}
  \Tree
    [.8
      [.5 [.3 [.1 [.0 ]] [.4 ]] [.7 [. 6 ]]]
      [.10 [.9 ] [.14 ]]
    ]
\end{tikzpicture}
\end{center}

```

```

\newpage
\section{Augmenting AVL Trees [22 marks]}
\subsection{1. Describe all information that will be stored in the nodes.}
Assume each person has two same node to be represents and there are  $n$  people
total with different start-point or end-point. I designed an AVL containd two
part: \textbf{left part} and \textbf{right part}. \textbf{right part} is an AVL
tree which has  $n$  nodes and ranked by start-point and \textbf{left part} is
an AVL tree which has  $n$  nodes and ranked by end-point. The root of each part
AVL tree is associated by initial root S-root.

```

```

\[\text{format}\ \text{nodes}\ \text{in}\ \text{the}\ \text{tree}\ :\
\begin{cases}
  \text{StartPoint}:\ \text{The}\ \text{endpoint}\ \text{of}\ \text{the}\ \text{interval.}\\
  \text{EndPoint}:\ \text{The}\ \text{start}\ \text{and}\ \text{end}\ \text{points}\ \text{of}\ \text{the}\ \text{interval.}\\
  \text{left}:\ \text{Pointers}\ \text{to}\ \text{the}\ \text{left}\ \text{child}\ \text{of}\ \text{the}\ \text{node.}\\
  \text{right}:\ \text{Pointers}\ \text{to}\ \text{the}\ \text{right}\ \text{child}\ \text{of}\ \text{the}\ \text{node.}
\end{cases}

```

```

\par The Rough drawing like this:

```

```

\begin{center}
  \begin{tikzpicture}
    \Tree
      [. \node[draw, circle]{S-root};
        [. \node[draw, rectangle]{left part(ranked by start-point)}; ]
        [. \node[draw, rectangle]{right part(ranked by end-point)}; ]
      ]
  \end{tikzpicture}
\end{center}

```

```

\subsection{2. Provide pseudo-code for each required operation.}

```

```

\subsubsection{\textbf{insert}(S, x, y):}

```

```

\begin{algorithm}
  \caption{insert(S, x, y)}
  \begin{algorithmic}[1]
    \IF{root == NULL}
      \STATE root = creatNewNode(0, 0)\qquad\COMMENT{creat an initial state}
    \ENDIF
    \STATE root\rightarrow left = insertNodeLeft(root\rightarrow left, x,
y)
    \STATE root\rightarrow right = insertNodeRight(root\rightarrow right,
x, y)
  \end{algorithmic}
\end{algorithm}

```

```

\begin{algorithm}
  \caption{insertNodeLeft(node, x, y)}
  \begin{algorithmic}[1]
    \IF{node == NULL}
      \STATE return creatNewNode(x, y)
    \ENDIF

    \IF{node\rightarrow StartPoint < x}
      \STATE node\rightarrow left = insertNodeLeft(node\rightarrow left, x,
y)
    \ENDIF
  \end{algorithmic}
\end{algorithm}

```

```

\ELSIF{node$\rightarrow$StartPoint$>$ x}
\STATE node$\rightarrow$right = insertNodeLeft(node$\rightarrow$right,
x, y)
\ELSE
\IF{node$\rightarrow$EndPoint $>$ y}
\STATE insertNodeLeft(node$\rightarrow$right, x, y)
\ELSIF{node$\rightarrow$EndPoint $<$ y}
\STATE insertNodeLeft(node$\rightarrow$left, x, y)
\ELSE
\STATE return node
\ENDIF
\ENDIF
\STATE return balance(node) \quad \COMMENT{keep it always an AVL tree}
\end{algorithmic}
\end{algorithm}
\begin{algorithm}
\caption{insertNodeRight(node, x, y)}
\begin{algorithmic}[1]
\IF{node == NULL}
\STATE return creatNewNode(x, y)
\ENDIF

\IF{node$\rightarrow$EndPoint $<$ y}
\STATE node$\rightarrow$left = insertNodeRight(node$\rightarrow$left, x,
y)
\ELSIF{node$\rightarrow$EndPoint $>$ y}
\STATE node$\rightarrow$right = insertNodeRight(node$\rightarrow$right,
x, y)
\ELSE
\IF{node$\rightarrow$StartPoint $>$ x}
\STATE node$\rightarrow$right = insertNodeRight(node$\rightarrow$right,
x, y)
\ELSIF{node$\rightarrow$StartPoint $<$ x}
\STATE node$\rightarrow$left = insertNodeRight(node$\rightarrow$left, x,
y)
\ELSE
\STATE return node
\ENDIF
\ENDIF
\STATE return balance(node) \quad \COMMENT{keep it always an AVL tree}
\end{algorithmic}
\end{algorithm}
\newpage
\subsubsection{\textbf{delete(S, x, y):}}
\begin{algorithm}
\caption{delete(S, x, y)}
\begin{algorithmic}[1]
\IF{root == NULL}
\STATE root = NULL\quad\COMMENT{The AVL is empty}
\ENDIF
\STATE root$\rightarrow$left = deleteNodeLeft(root$\rightarrow$left, x,
y)
\STATE root$\rightarrow$right = deleteNodeRight(root$\rightarrow$right,
x, y)
\end{algorithmic}
\end{algorithm}
\begin{algorithm}
\caption{deleteNodeRight(node, x, y)}
\begin{algorithmic}[1]
\IF{node == NULL}
\STATE return NULL
\ENDIF

\IF{node$\rightarrow$EndPoint $<$ y}

```

```

\STATE node$\rightarrow$left = deleteNodeRight(node$\rightarrow$left, x,
y)
\ELSIF{node$\rightarrow$EndPoint$>$ y}
\STATE node$\rightarrow$right = deleteNodeRight(node$\rightarrow$right,
x, y)
\ELSE
\IF{node$\rightarrow$StartPoint $>$ x}
\STATE node$\rightarrow$right = deleteNodeRight(node$\rightarrow$right,
x, y)
\ELSIF{node$\rightarrow$StartPoint $<$ x}
\STATE node$\rightarrow$left = deleteNodeRight(node$\rightarrow$left, x,
y)
\ELSE
\IF{node has no child}
\STATE free(node)
\ELSIF{one child (no matter is left or right)}
\STATE free(node)
\STATE return balance(child)\quad\COMMENT{if necessary to keep it as
AVL tree}
\ELSE
\STATE let x = successor(node)\quad \COMMENT{the most left child node
of the node$\rightarrow$right}
\STATE set some value to store the \textbf{successor}'s parameters\
\quad\COMMENT{StartPoint \& EndPoint}
\STATE node$\rightarrow$right = deleteNodeRight(node$\rightarrow$right,
x, y)
\STATE set the \textbf{node's} parameters to the \textbf{successor}'s
parameters
\ENDIF
\ENDIF
\ENDIF
\STATE return balance(node) \quad \COMMENT{keep it always an AVL tree}
\end{algorithmic}
\end{algorithm}
\begin{algorithm}
\caption{deleteNodeLeft(node, x, y)}
\begin{algorithmic}[1]
\IF{node == NULL}
\STATE return NULL
\ENDIF

\IF{node$\rightarrow$StartPoint $<$ x}
\STATE node$\rightarrow$left = deleteNodeLeft(node$\rightarrow$left, x,
y)
\ELSIF{node$\rightarrow$StartPoint$>$ x}
\STATE node$\rightarrow$right = deleteNodeLeft(node$\rightarrow$right,
x, y)
\ELSE
\IF{node$\rightarrow$EndPoint $>$ y}
\STATE node$\rightarrow$right = deleteNodeLeft(node$\rightarrow$right,
x, y)
\ELSIF{node$\rightarrow$EndPoint $<$ y}
\STATE node$\rightarrow$left = deleteNodeLeft(node$\rightarrow$left, x,
y)
\ELSE
\IF{node has no child}
\STATE free(node)
\ELSIF{one child (no matter is left or right)}
\STATE free(node)
\STATE return balance(child)\quad\COMMENT{if necessary to keep it as
AVL tree}
\ELSE
\STATE let x = successor(node)\quad \COMMENT{the most left child node
of the node$\rightarrow$right}

```



```

\STATE set some value to store the \textbf{successor}'s parameters\
qqquad\COMMENT{StartPoint \& EndPoint}
\STATE node$\rightarrow$right = deleteNodeLeft(node$\rightarrow$right,
x, y)
\STATE set the \textbf{node's} parameters to the \textbf{successor}'s
parameters
\ENDIF
\ENDIF
\ENDIF
\STATE return balance(node) \qqquad \COMMENT{keep it always an AVL tree}
\end{algorithmic}
\end{algorithm}
\newpage
\subsubsection{\textbf{meetAll}(S, l, h):{}}
\begin{algorithm}
\caption{bool meetAll(S, l, h)}
\begin{algorithmic}[1]
\IF{S is empty}
\STATE return false
\ENDIF

\STATE let x = root$\rightarrow$left
\WHILE{x$\rightarrow$right != NULL}
\IF{x$\rightarrow$StartPoint $\leq$ h}
\STATE x = x$\rightarrow$right
\ENDIF
\ENDWHILE

\IF{x$\rightarrow$StartPoint $\leq$ h}
\STATE let y = root$\rightarrow$right
\WHILE{y$\rightarrow$left != NULL}
\IF{y$\rightarrow$EndPoint $\geq$ l}
\STATE y = y$\rightarrow$left
\ENDIF
\ENDWHILE
\IF{y$\rightarrow$EndPoint $\geq$ l}
\STATE return true
\ELSE
\STATE print the \textbf{y's StartPoint and EndPoint}
\ENDIF
\ELSE
\STATE print the \textbf{x's StartPoint and EndPoint}
\ENDIF
\STATE return false
\end{algorithmic}
\end{algorithm}
\subsection{Justify why your algorithms are correct and why they achieve the
required time bound.}
\subsubsection{Insertion and Deletion:}
Each operation contains two part of operations follow the standard AVL tree
insertion and deletion procedures with additional steps to some logical judgment
sentences and perform balancing if required. Since AVL tree insertion and
deletion have time complexity  $O(\log n)$ , my additional operations involve only
constant-time updates and I repeat the operation twice in each final methods,
the overall complexity remains  $O(2\log n)$  (which is also in  $O(\log n)$ ).
\subsubsection{meetAll:}
The meetAll operation firstly recursively traverses the left part AVL tree to
check if every StartPoints are before the given \textbf{h}. If all StartPonts
good, then check if all Endpoints are after the given \textbf{l}. If one of them
is unsuitable then quit the program and print the current interval. The
traversal of the tree is done in a manner similar to binary search, resulting in
a time complexity of  $O(\log n)$ (Since the worst condition is  $O(2\log n)$ ).

\end{document}

```